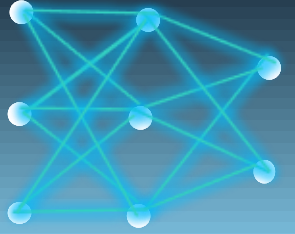




Deep Learning

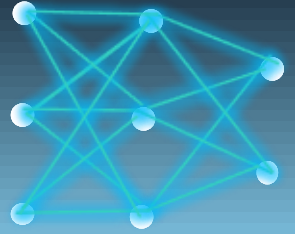
Dr. Shay Maymon

Natural Language Processing



Languages represent an example of sequential data

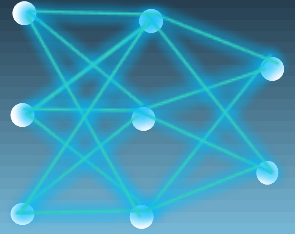
NLP Tasks



- **Text Classification**

- Sentiment Analysis (positive/ negative/neutral)
- Spam Detection
- Topic Classification
- Language Identification

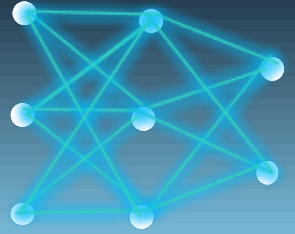
NLP Tasks



- **Sequence Labeling**

- Part-of-speech tagging
- Named entity Recognition (NER) (Identifying people, locations, time,...)
- Chunking
- Morphological Tagging

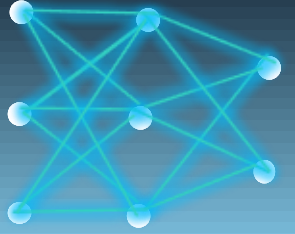
NLP Tasks



- **Text Generation**

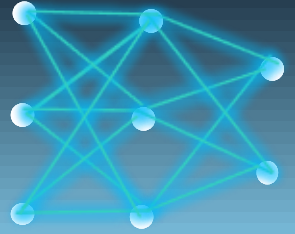
- Machine Translation
- Text Summarization
- Story Generation
- Code Generation
- Question Answering

Natural Language Processing



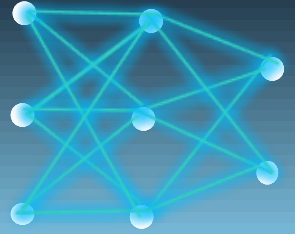
The first challenge is to convert the words into a numerical representation

Tokenization



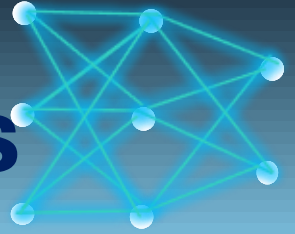
- Tokenization is the process of splitting text into smaller pieces called **tokens** so a computer model can process it
- Models do not read text exactly as humans do

Word Level Tokenization – Advantages

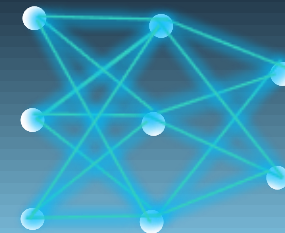


- **Intuitive Representation**
- **Shorter Sequences** – reduce computational cost
- **Fast Preprocessing** – Splitting on whitespace and punctuation is computationally simple

Word Level Tokenization – Disadvantages



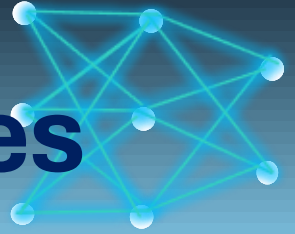
- **Huge Vocabulary** – even a moderate corpus may contain millions of unique words.
- **Poor handling of morphology** – Related words are treated independently (run, runs, running, runner, runners)
- Misspellings become unknown words
- **Out-of_vocabulary (OOV) problem** – If the model encounters an unseen word during inference, it cannot represent it



Character Level Tokenization

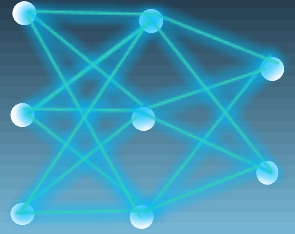
- Work at the level of characters
- Example – Learning -> ‘L’, ‘e’, ‘a’, ‘r’, ‘n’, ‘i’, ‘n’, ‘g’
- The dictionary will comprise
 - Upper-case letters
 - Lower-case letters
 - Numbers
 - Punctuation
 - Spaces

Character Level Tokenization - Advantages

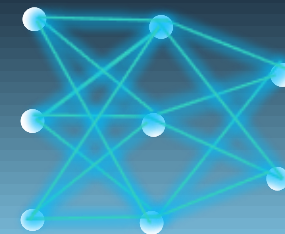


- **Small vocabulary size** – A character vocabulary is tiny
 - English Alphabet: 26 lowercase letters, 26 uppercase letters, 10 digits, punctuation, whitespace, symbols (80-120 characters)
- **No Out-of-vocabulary (OOV) problem** – A character tokenizer can represent any word, including:
 - Misspellings
 - new slang
 - rare names
 - technical terms
 - new languages

Character Level Tokenization - Disadvantages



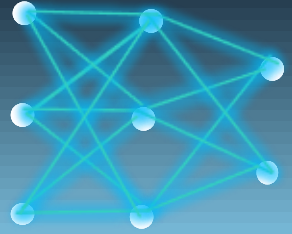
- **Much longer sequences** – Longer sequences mean
 - Higher computational cost
 - More memory
- **Harder to Learn Semantic Meaning** – Character models often need deeper networks, more training data, more computation
- **Difficult Long-range dependencies** – important relationships become farther apart



Sub-words Tokenization

- Combine the **benefits** of **character-level** and **word-level** representations
- Small groups of characters
- Might include
 - **common words** in their entirety
 - **fragments of longer words**
 - **individual characters** that can be assembled into less common words

Sub-words Tokenization



Unbelievable

Un+believe+able

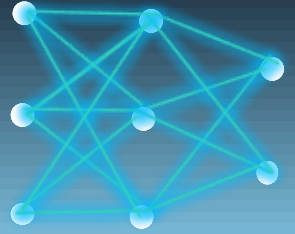
hyperparameterization

hyper + parameter + ization

Antidisestablishmentarianism

anti + dis + establishment + arian + ism

Sub-words Tokenization - Advantages



- **Handles unknown words better**

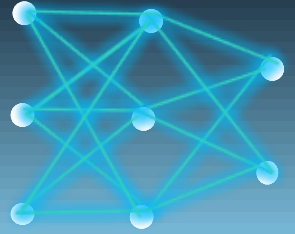
Example: I bought a solarbackpack

Word-level Tokenizer: I | bought | a | <UNK> |

Sub-word Tokenizer: I | bought | a | solar | backpack |

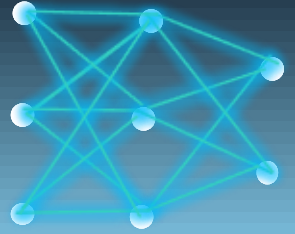
- **Smaller vocabulary than word-level tokenization**
 - A word-level tokenizer may need millions of words
 - Subwords tokenization can cover many languages with tens of thousands of tokens

Sub-words Tokenization - Advantages

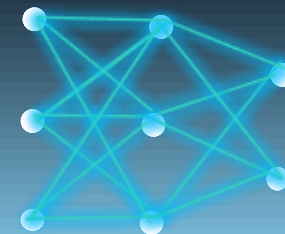


- **Allows to process other kinds of sequences** (i.e., computer code, images)
- **Variations of the same word can have related representations** ('cook', 'cooks', 'cooking', 'cooked', 'cooker')
- **Works well for rare words**
Rare names, technical terms, typos, can still be represented meaningfully

Sub-words Tokenization - Advantages



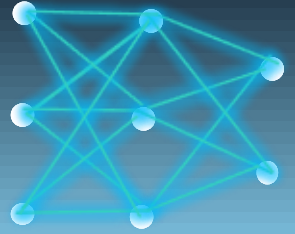
- **Can struggle with morphology and meaning**
Subword splits are statistical, not always linguistic. A tokenizer may split a word in a way that ignores prefixes, roots, or suffixes



Sub-words Tokenization

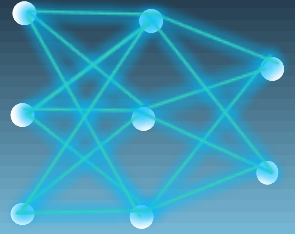
- Subword tokenization is popular because it gives a strong practical tradeoff: it avoids unknown words while keeping vocabulary size manageable
- Its main downside is that it is still a statistical compression trick, not a perfect representation of language meaning

Byte Pair Encoding



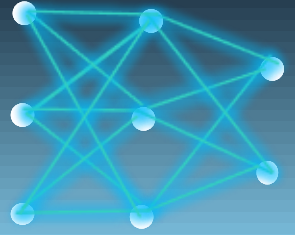
- Starts with the individual characters and iteratively merges them into longer strings
- The most frequently occurring adjacent pairs of tokens are replaced with a new token
- Initially, the number of tokens is equal to the number of characters, which is relatively small

Byte Pair Encoding



- As tokens are formed, the **total number of tokens increases**, and if this is continued long enough, the tokens will eventually correspond to the set of words in the text
- The **total number of tokens** is generally fixed in advance, as a **compromise** between **character-level** and **word-level** representations

Byte Pair Encoding



Peter Piper picked a peck of pickled peppers

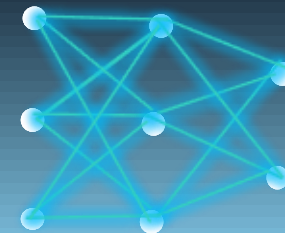
Peter Piper picked a peck of pickled peppers

Peter Piper picked a peck of pickled peppers

Peter Piper picked a peck of pickled peppers

Peter Piper picked a peck of pickled peppers

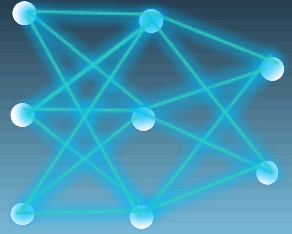
Peter Piper picked a peck of pickled peppers



From Tokens to Numbers

- Tokenization breaks text into smaller pieces called **tokens**
- However, computers cannot directly understand words, so we need to convert these tokens into numbers

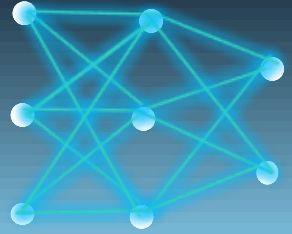
From Tokens to Numbers



I love natural language processing
["I", "love", "natural", "language", "processing"]

Token	Number
I	1
love	2
natural	3
language	4
processing	5

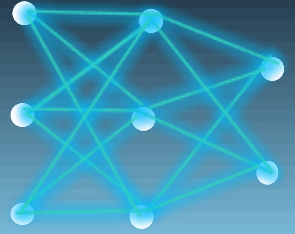
From Tokens to Numbers



- The model may think that **language** is closer to **processing** than to **I**

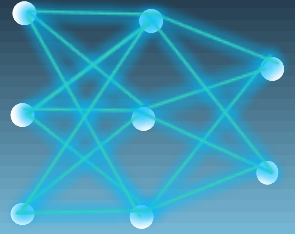
Token	Number
I	1
love	2
natural	3
language	4
processing	5

One-hot Encoding



- Define a fix dictionary of words
- Introduce vectors of length equal to the size of the dictionary
- ‘**one hot**’ representation for each word

One-hot Encoding



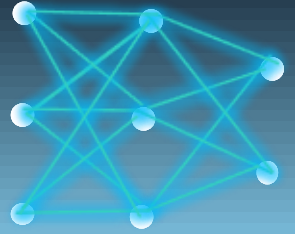
One-Hot Encoding for Word Representation

Vocabulary			One-Hot Vectors				
			1	2	3	4	
1		cat	1	0	0	0	
2		dog	0	1	0	0	
3		fish	0	0	1	0	
4		bird	0	0	0	1	

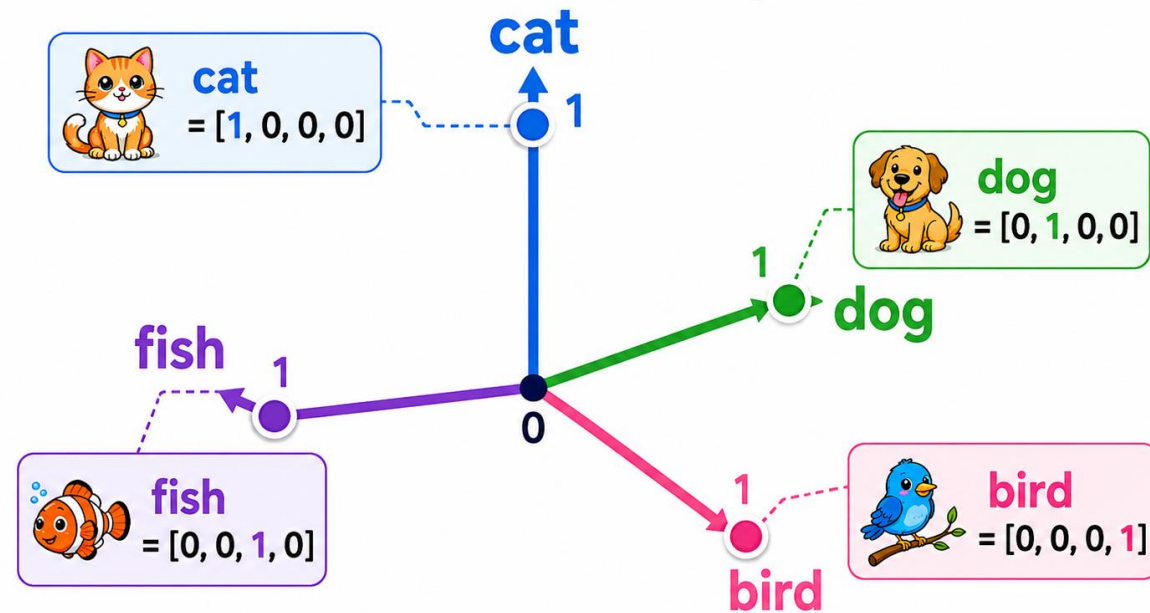


Each word is represented by a vector with one **1** and the rest **0**s.

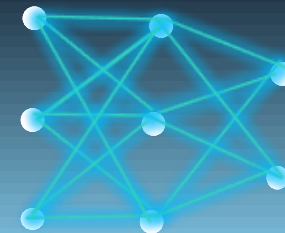
One-hot Encoding



One-Hot Encoding as Axes



In one-hot encoding, each word gets its own axis.
The represented word has value **1** on its axis and **0** on all others.



One-hot Encoding - Cons

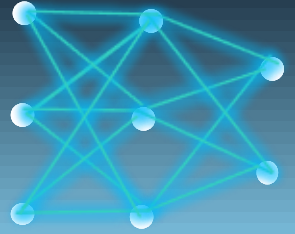
- A realistic dictionary might have **several hundred thousands** entries
- Does not capture **similarities** or **relationships** between words

From Word Vectors to Text Representation



- **One-hot encoding** represents one token at a time
- Each token is represented using a vector based on the vocabulary
- How do we represent a full sentence?

Bag of Words

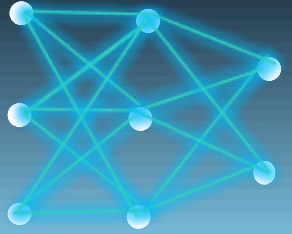


- The words are drawn independently from the same distribution

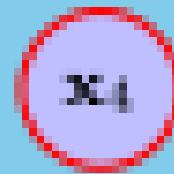
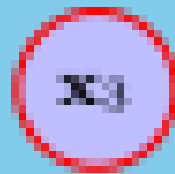
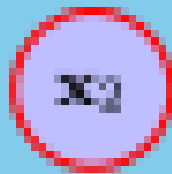
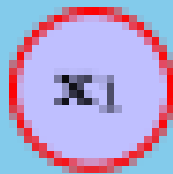
$$p(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N) = \prod_{n=1}^N p(\mathbf{x}_n)$$

- The distribution can be represented as a simple table listing the probabilities of each of the possible states of $\mathbf{x}$ (corresponding to the dictionary of words or tokens)

Bag of Words

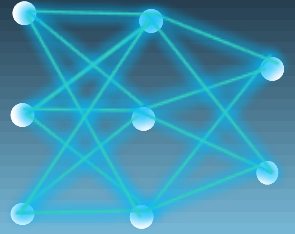


$$p(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N) = \prod_{n=1}^N p(\mathbf{x}_n)$$



...

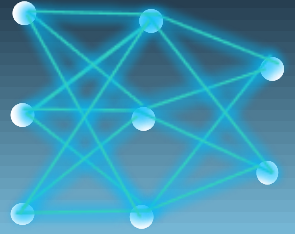
Bag of Words – Maximum Likelihood



- The ML solution for this model is obtained simply by setting each of these probabilities to the fraction of times that the word occurs in the training set

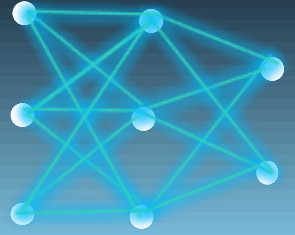
$$p(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N) = \prod_{n=1}^N p(\mathbf{x}_n)$$

Bag of Words



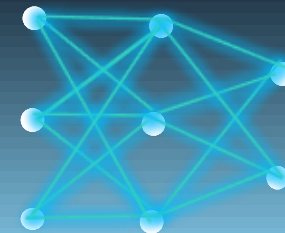
- It is called **Bag of Words** because it completely ignores the ordering of the words
- Can be used for constructing a simple text classifier
- Sentiment Analysis for a restaurant review (**Positive**/**Negative**)

Naive Bayes Classifier



- This classifier assumes that the words are independent within each class $\mathcal{C}_k$, but with a different distribution for each class

$$p(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N | \mathcal{C}_k) = \prod_{n=1}^N p(\mathbf{x}_n | \mathcal{C}_k)$$



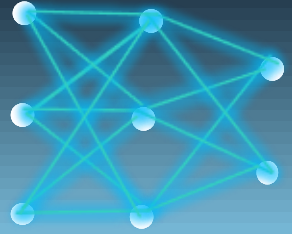
Naive Bayes Classifier

- Given prior class distribution $p(\mathcal{C}_k)$, the posterior class probabilities for a new sequence are given by:

$$p(\mathcal{C}_k | \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N) \propto p(\mathcal{C}_k) \prod_{n=1}^N p(\mathbf{x}_n | \mathcal{C}_k)$$

- Both the class-conditional densities $p(\mathbf{x}_n | \mathcal{C}_k)$ and the prior probabilities $p(\mathcal{C}_k)$ can be estimated using frequencies from the training data set

Bag of Words



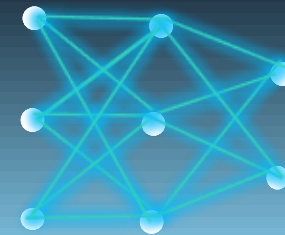
Example sentences

Positive: “I love this movie”

Negative: “I hate this movie”

Sentence	I	love	hate	this	movie
“I love this movie”	1	1	0	1	1
“I hate this movie”	1	0	1	1	1

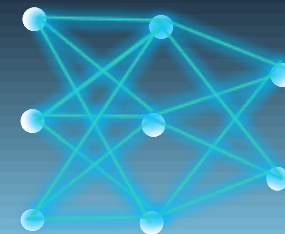
The model learns that words like **love** often signal positive sentiment, while words like **hate** often signal negative sentiment



A major limitation of **Bag of Words** is that it completely ignores word order

“Dog bites man” vs. “Man bites dog”

“The movie was **bad**, not **good**” vs. “The movie was **good**, not **bad**”



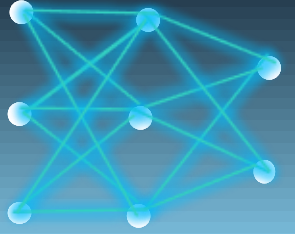
Autoregressive Models

- The distribution over the sequence of words can be decomposed into a product of conditional distribution in the form

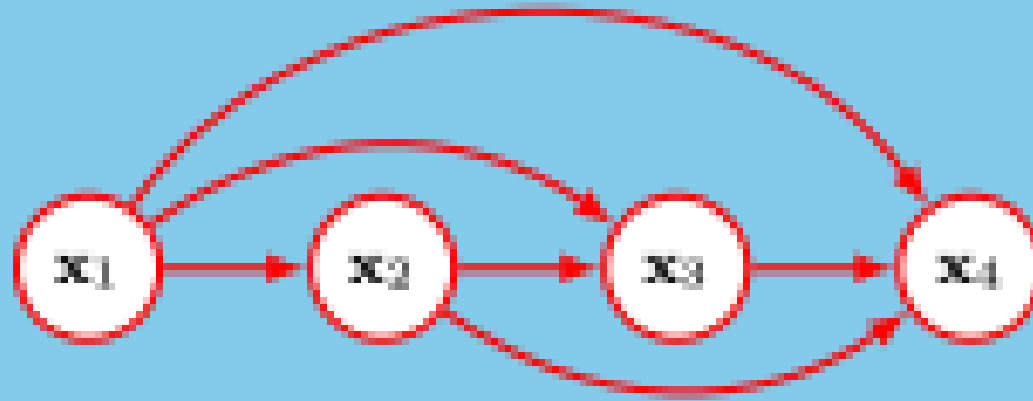
$$p(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N) = \prod_{n=1}^N p(\mathbf{x}_n | \mathbf{x}_{n-1}, \mathbf{x}_{n-2}, \dots, \mathbf{x}_1)$$

- We could represent each term by a table whose entries are estimated using simple frequency counts from the training set
- The size of these tables grows exponentially with the length of the sequence

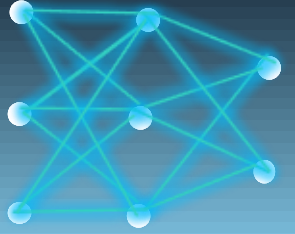
Autoregressive Models



$$p(x_1, x_2, \dots, x_N) = \prod_{n=1}^N p(x_n | x_{n-1}, x_{n-2}, \dots, x_1)$$



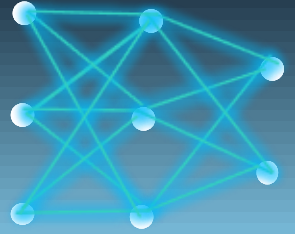
Autoregressive Models



- Each of the conditional distributions is independent of all previous observations except the L most recent words

$$p(\mathbf{x}_n | \mathbf{x}_{n-1}, \mathbf{x}_{n-2}, \dots, \mathbf{x}_1) = p(\mathbf{x}_n | \mathbf{x}_{n-1}, \mathbf{x}_{n-2}, \dots, \mathbf{x}_{n-L})$$

A Tri-gram Model

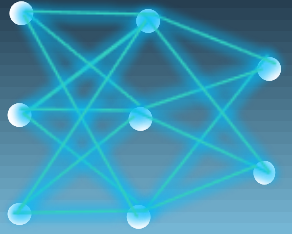


- Involve triplets of adjacent words ($L = 2$)

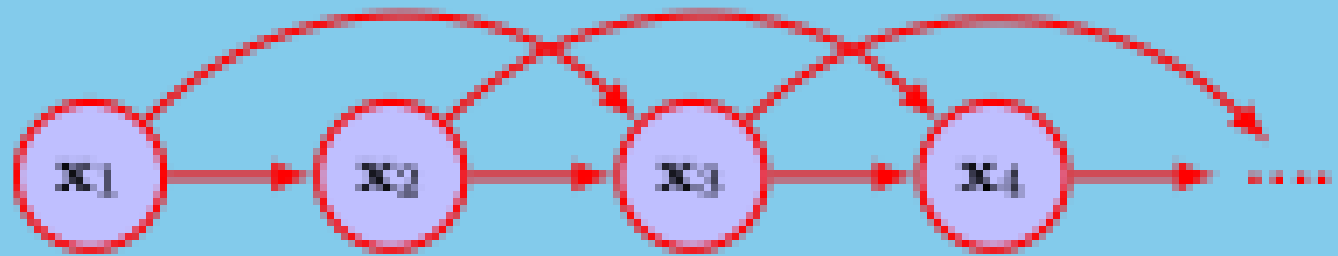
$$p(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N) = p(\mathbf{x}_1)p(\mathbf{x}_2|\mathbf{x}_1) \prod_{n=3}^N p(\mathbf{x}_n|\mathbf{x}_{n-1}, \mathbf{x}_{n-2})$$

- Each of $p(\mathbf{x}_n|\mathbf{x}_{n-1}, \mathbf{x}_{n-2})$ can be represented as tables whose values are estimated from the statistics of the triplets of successive words drawn from a training corpus

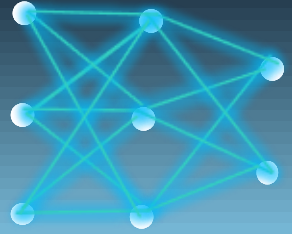
A Tri-gram Model



$$p(x_1, x_2, \dots, x_N) = p(x_1)p(x_2|x_1) \prod_{n=3}^N p(x_n|x_{n-1}, x_{n-2})$$

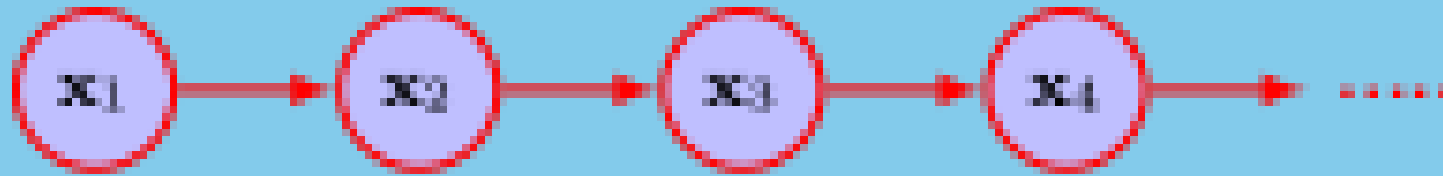


A Bi-gram Model

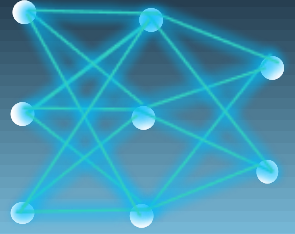


- Depends on pairs of adjacent words ($L = 1$)

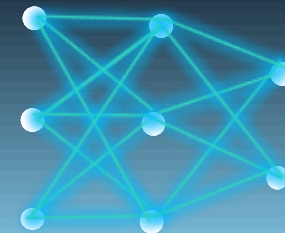
$$p(x_1, x_2, \dots, x_N) = p(x_1) \prod_{n=2}^N p(x_n | x_{n-1})$$



Why Smoothing Is Needed in N-gram Models



- Maximum likelihood assigns **zero probability** to unseen n -grams
- Real test data may contain valid phrases missing from training data
- A single zero probability makes the **entire test-set probability zero**

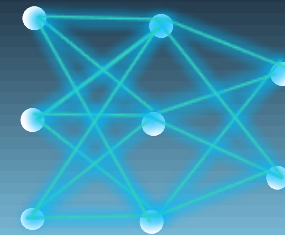


Laplace (Add-One) Smoothing

- Add **1 to every n-gram count** before computing probabilities
- This gives unseen n-grams a **non-zero probability**
- For unigrams:

$$p_{Laplace}(w_i) = \frac{c_i}{N}$$

- Simple and useful as a baseline, but **not strong enough for modern n-gram models**

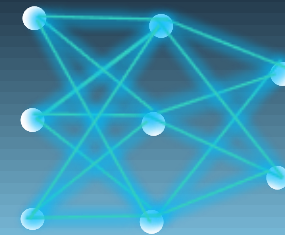


Laplace (Add-One) Smoothing

- Add **1** to **every n-gram count** before computing probabilities
- This gives unseen n-grams a **non-zero probability**
- For unigrams:

$$p_{Laplace}(w_i) = \frac{c_i + 1}{N}$$

- Simple and useful as a baseline, but **not strong enough for modern n-gram models**



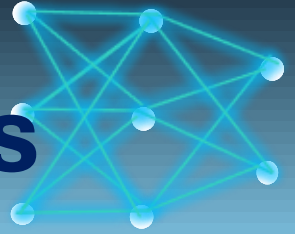
Laplace (Add-One) Smoothing

- Add **1 to every n-gram count** before computing probabilities
- This gives unseen n-grams a **non-zero probability**
- For unigrams:

$$p_{Laplace}(w_i) = \frac{c_i + 1}{N + V}$$

- N is the total word tokens, V is the vocabulary size
- Simple and useful as a baseline, but **not strong enough for modern n-gram models**

Laplace (Add-One) Smoothing – Bi-grams

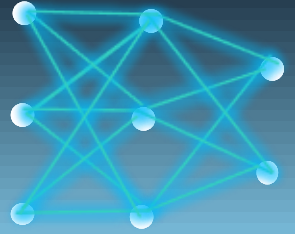


$$p_{ML}(w_n|w_{n-1}) = \frac{c(w_{n-1}w_n)}{c(w_{n-1})}$$

$$p_{Laplace}(w_n|w_{n-1}) = \frac{c(w_{n-1}w_n) + 1}{\sum_w c(w_{n-1}w) + 1} = \frac{c(w_{n-1}w_n) + 1}{c(w_{n-1}) + V}$$

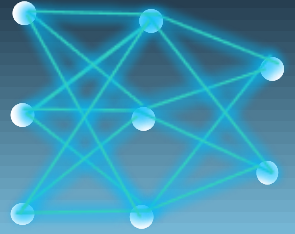
- N is the total word tokens, V is the vocabulary size
- Laplace smoothing is easy and prevents zeros, but it is usually too crude for serious n-gram language modeling

Synthesize Novel Text



- If we provide the first and second words in a sequence, we can sample from the tri-gram statistics $p(\mathbf{x}_n | \mathbf{x}_{n-1}, \mathbf{x}_{n-2})$ to generate the third word...
- The resulting text will be **incoherent**
- **High-quality** text models must take account of the **long-range dependencies** in language
- We cannot increase the value of L - it is **prohibitively expensive** to go much beyond tri-gram modls

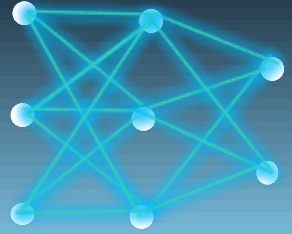
Examples



Randomly generated from four n-gram models computed from Shakespeare's work

- | | |
|------|--|
| 1 | -To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have |
| gram | -Hill he late speaks; or! a more to leg less first you enter |
| 2 | -Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow. |
| gram | -What means, sir. I confess she? then all sorts, he is trim, captain. |
| 3 | -Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done. |
| gram | -This shall forbid it should be branded, if renown made it empty. |
| 4 | -King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in; |
| gram | -It cannot be but so. |

Examples



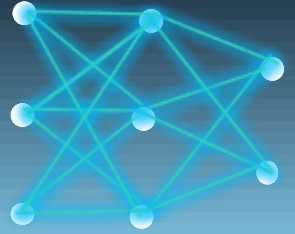
Randomly generated from three n-gram models
computed from 40 million words of the Wall Street
Journal

1
gram Months the my and issue of year foreign new exchange's september
were recession exchange new endorsed a acquire to six executives

2
gram Last December through the way to preserve the Hudson corporation N.
B. E. C. Taylor would seem to complete the major central planners one
point five percent of U. S. E. has already old M. X. corporation of living
on information such as more frequently fishing to keep her

3
gram They also point to ninety nine point six billion dollars from two hundred
four oh six three percent of the rates of interest stores as Mexico and
Brazil on market conditions

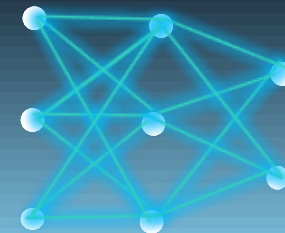
Examples



- unigram, bigram, and trigram models were trained on 38 million words from the Wall Street Journal newspaper
- The perplexity of each of these models was computed on a WSJ test set for unigrams, for bigrams, and for trigrams
- The table below shows the perplexity of the 1.5 million word test set according to each of the language models

	Unigram	Bigram	Trigram
Perplexity	962	170	109

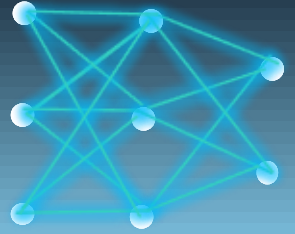
$$\text{perplexity}(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i|w_1 \dots w_{i-1})}}$$



n -grams have **poor scaling** with **sequence length** because they store completely general tables of conditional distributions

The number of parameters grows exponentially with L

Word Embedding

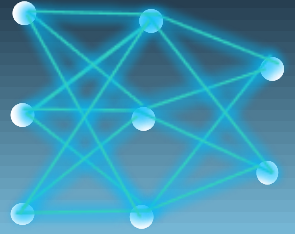


$$\mathbf{v}_n = E \mathbf{x}_n$$

$$\mathbf{v}_n \in \mathcal{R}^D \quad \mathbf{x}_n \in \mathcal{R}^K$$

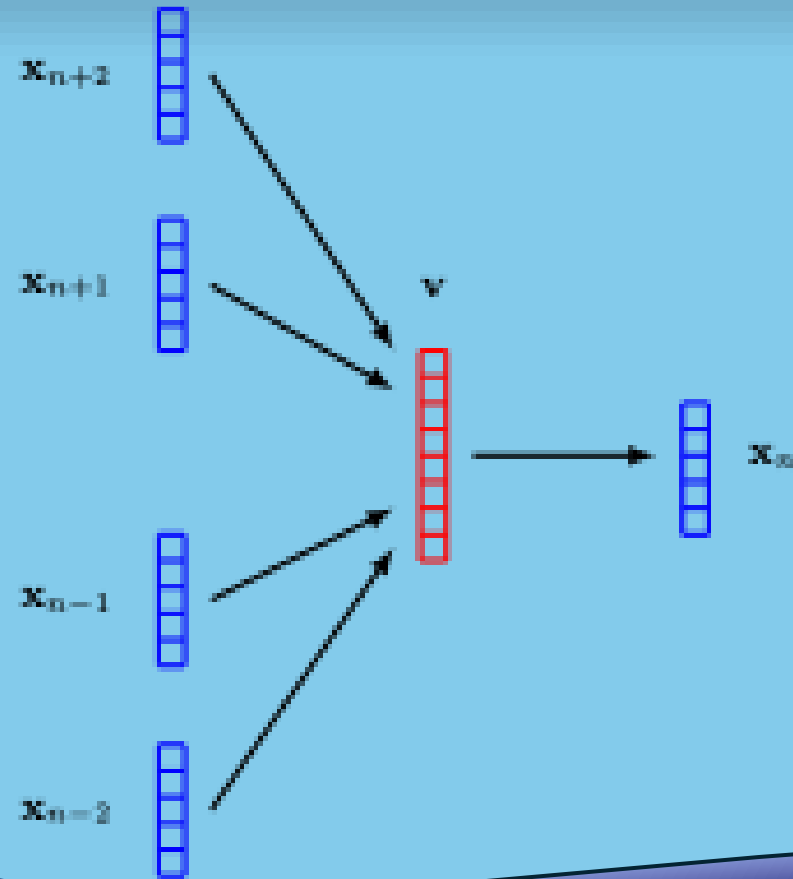
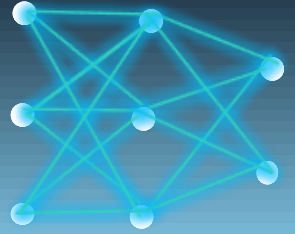
- The matrix $E \in \mathcal{R}^{D \times K}$ can be learned from a large data set of text

Word2Vec

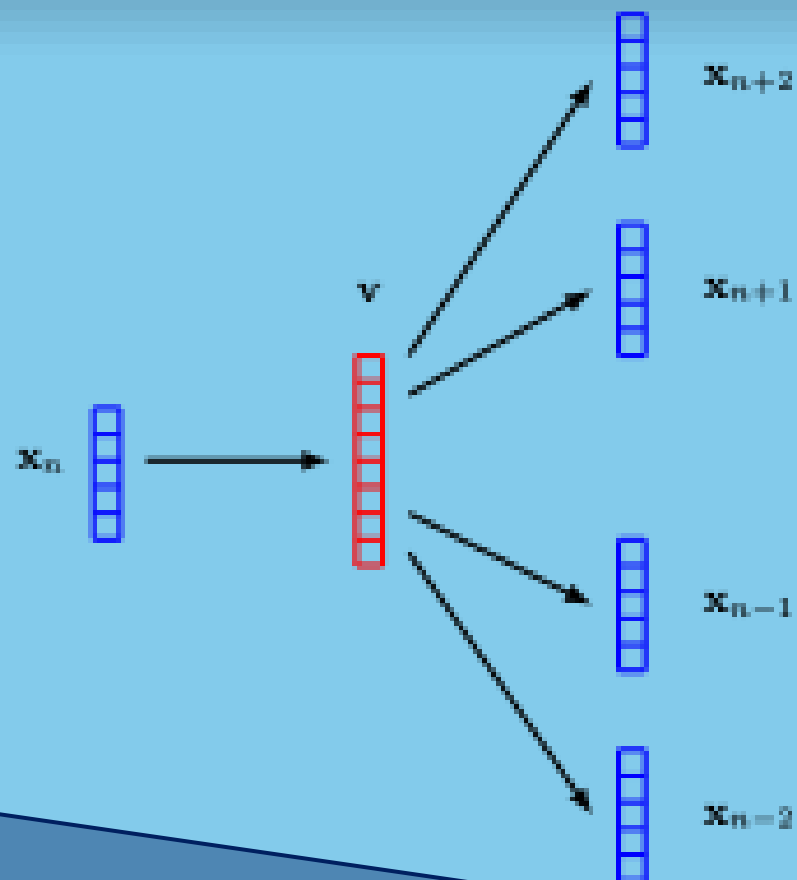
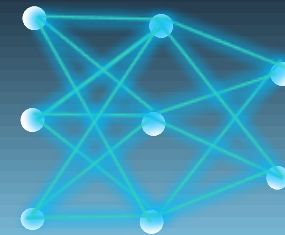


- Each sample in the training set is obtained by considering a window of M adjacent words in the text
- Two variants:
 - CBOW – Continuous Bag of Words
 - Skip-gram

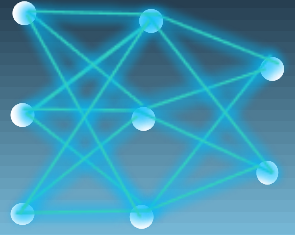
Continuous Bag of Words



Skip-grams

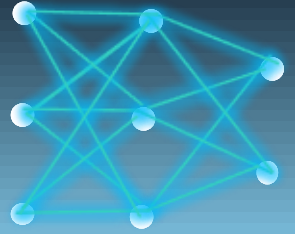


Self-supervised Learning



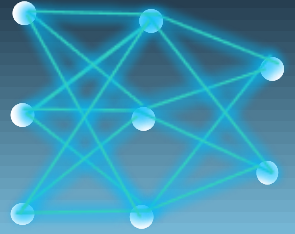
- The data consists of a large corpus of unlabelled text
- Many small windows of word sequences are drawn at random
- Labels are obtained from the text itself

The Embedding Matrix

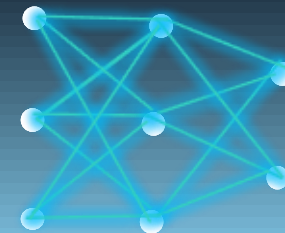


- Once the model is trained, the embedding matrix is usually obtained by the first-layer weight matrix

The embedding Space

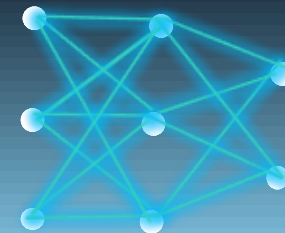


- Words that are semantically related are mapped to nearby positions in the embedding space
- Related words are more likely to occur with similar context words compared to unrelated words



The embedding Space - Example

- '**city**' and '**capital**' might occur with higher frequency as context for target words '**paris**' or '**London**' and less frequently as context for '**orange**' or '**polynomial**'
- The network can more easily predict the probability of the missing words if '**Paris**' and '**London**' are mapped to nearby embedding vectors

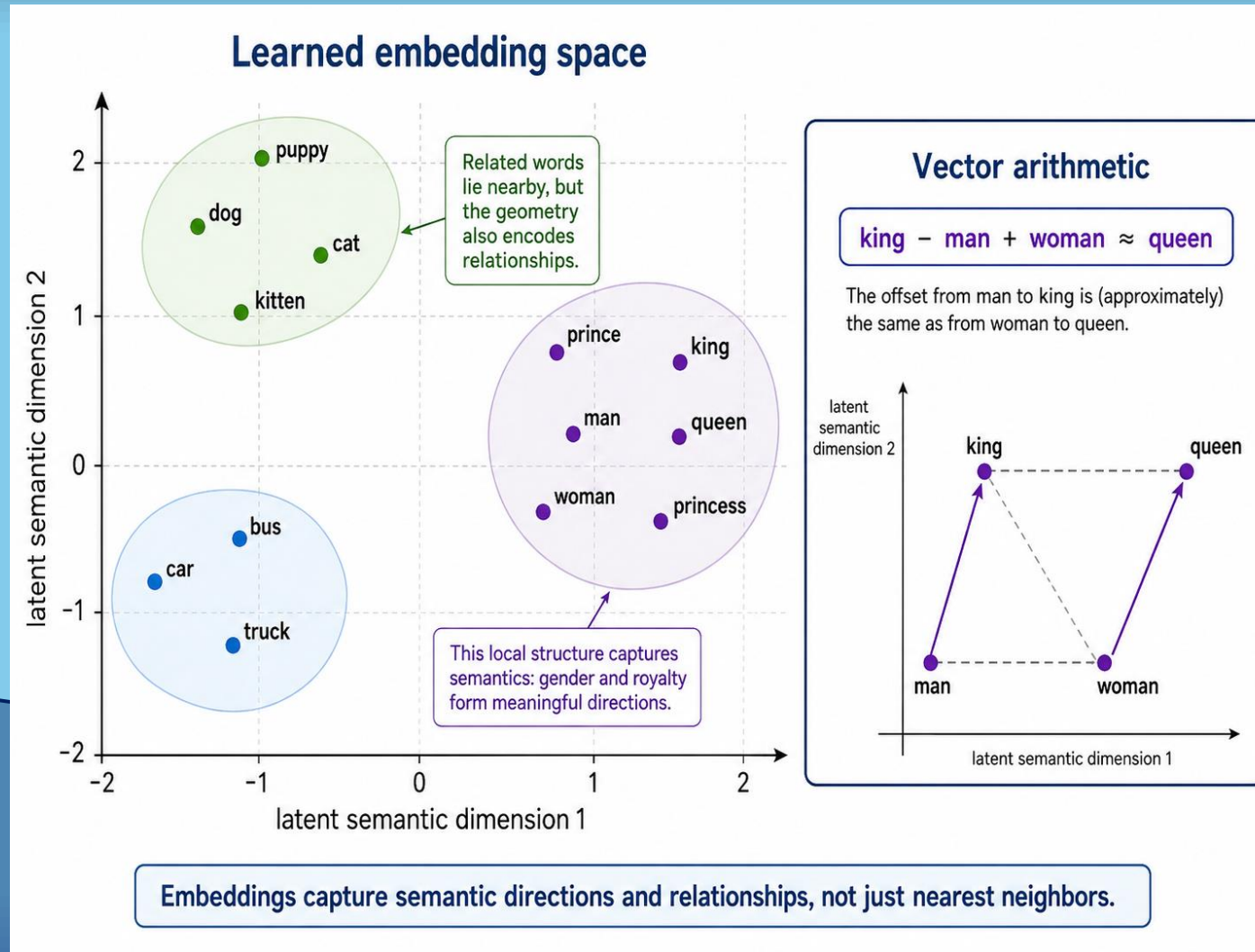
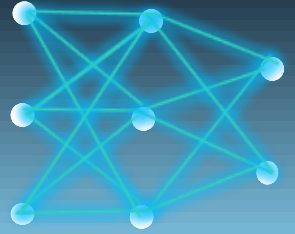


Learned Embedding Space

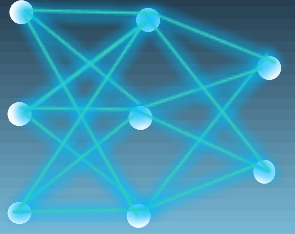
- The learned embedding space has a **richer semantic structure** than just the **proximity of related words**
- It allows for **simple vector arithmetic**

$$v(\textit{Paris}) - v(\textit{France}) + v(\textit{Italy}) \approx v(\textit{Rome})$$

Learned Embedding Space

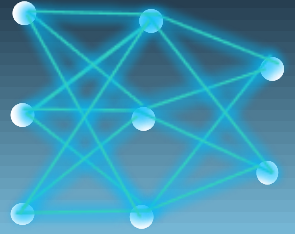


A standard Feedforward Neural Network



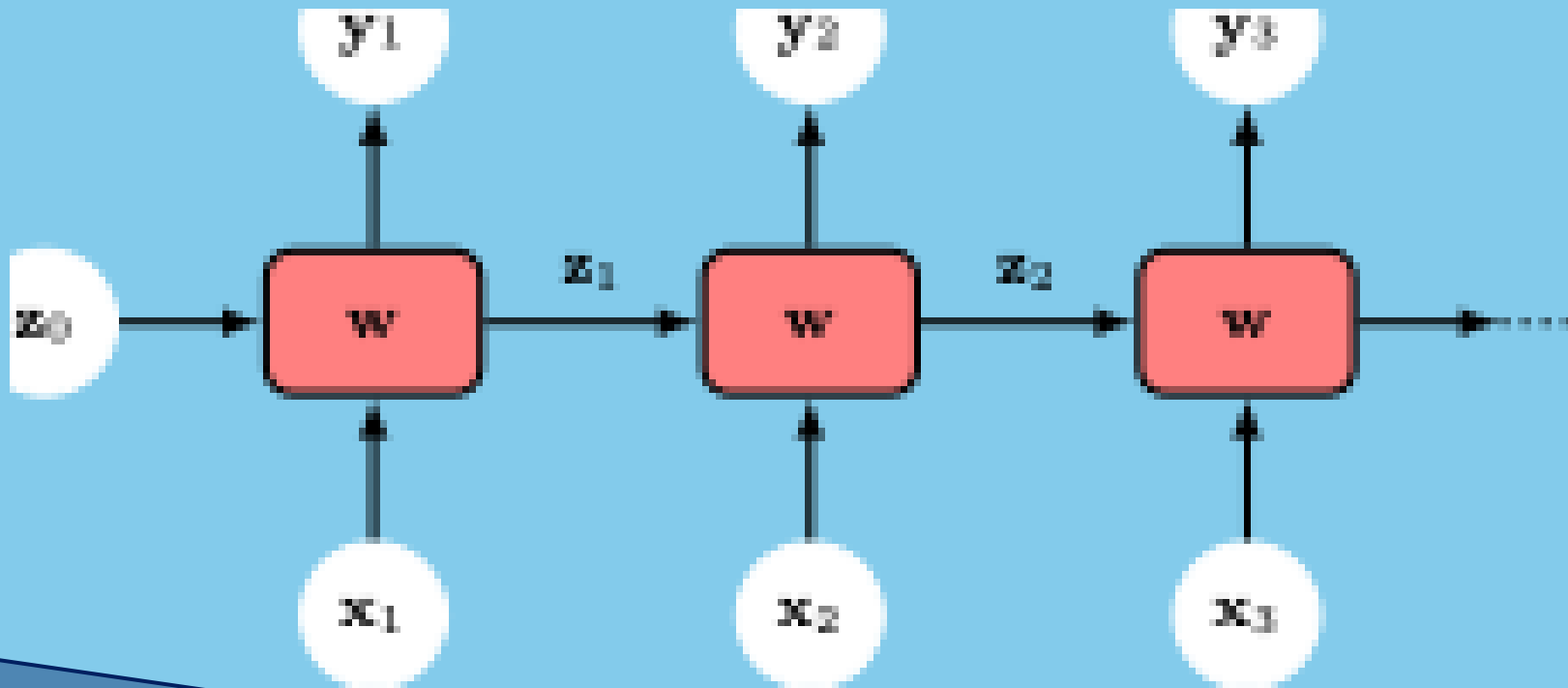
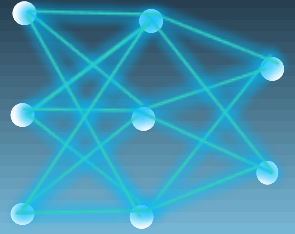
- A standard feedforward neural network has a fixed number of inputs and outputs
- We need to be able to handle variable-lengths sequences in the training and test sets
- Equivariance property

Recurrent Neural Networks (RNNs)

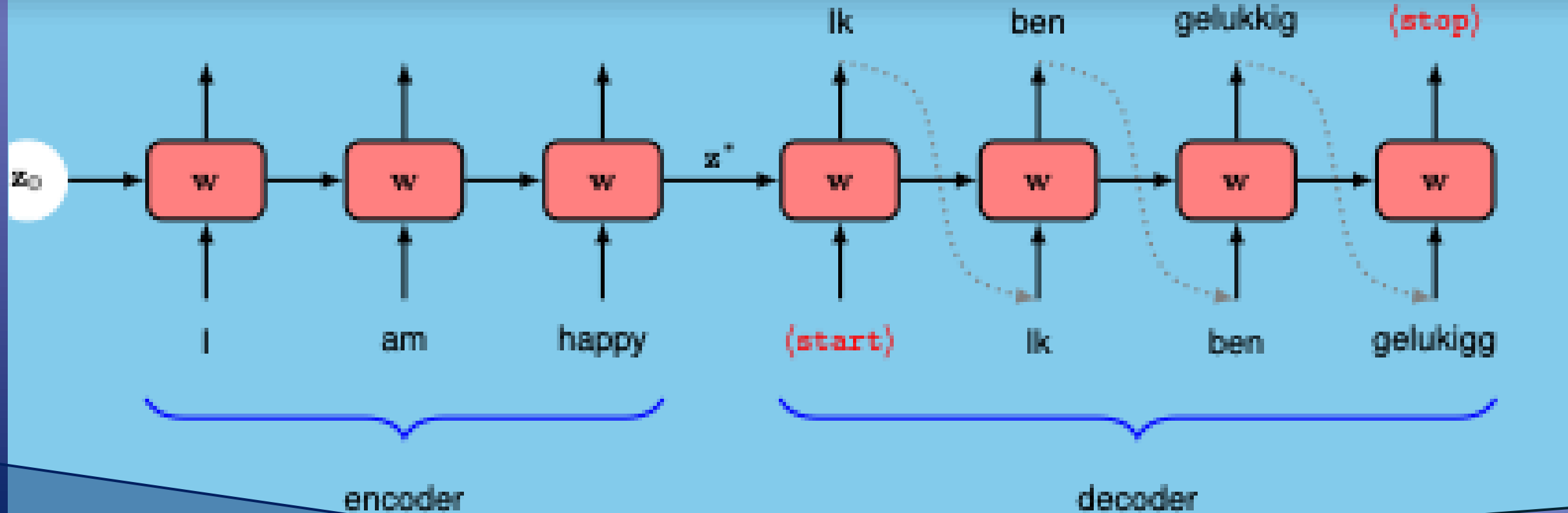


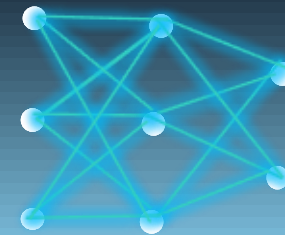
- Can borrow inspiration from the hidden Markov Model and introduce an explicit hidden variable $\mathbf{z}_n$ associated with each step n in the sequence
- The neural network takes as input both the current word $\mathbf{x}_n$ and the current hidden state $\mathbf{z}_{n-1}$ and produces an output word $\mathbf{y}_n$ as well as the next state $\mathbf{z}_n$ of the hidden variable
- We can chain together copies of this network, in which the weight values are shared across the copies

RNN



RNN used for Language Translation





Thank you